

TraGe: A Generic Packet Representation for Traffic Classification Based on Header-Payload Differences

Chungang Lin^{†‡||}, Yilong Jiang^{†‡||}, Weiyao Zhang[†], Xuying Meng^{†§*}, Tianyu Zuo[‡], Yujun Zhang^{†‡¶*}

[†]Institute of Computing Technology, Chinese Academy of Sciences, China.

[‡]University of Chinese Academy of Sciences, China.

[§]Purple Mountain Laboratories, China.

[¶]Nanjing Institute of InforSuperBahn, China.

{linchungang22s, jiangyilong23s, zhangweiyao17z, mengxuying, zuotianyu22s, nrcyujun}@ict.ac.cn

Abstract—Traffic classification has a significant impact on maintaining the Quality of Service (QoS) of the network. Since traditional methods heavily rely on feature extraction and large-scale labeled data, some recent pre-trained models manage to reduce the dependency by utilizing different pre-training tasks to train generic representations for network packets. However, existing pre-trained models typically adopt pre-training tasks developed for image or text data, which are not tailored to traffic data. As a result, the obtained traffic representations fail to fully reflect the information contained in the traffic, and may even disrupt the protocol information. To address this, we propose TraGe, a novel generic packet representation model for traffic classification. Based on the differences between the header and payload—the two fundamental components of a network packet—we perform differentiated pre-training according to the byte sequence variations (continuous in the header vs. discontinuous in the payload). A dynamic masking strategy is further introduced to prevent overfitting to fixed byte positions. Once the generic packet representation is obtained, TraGe can be fine-tuned for diverse traffic classification tasks using limited labeled data. Experimental results demonstrate that TraGe significantly outperforms state-of-the-art methods on two traffic classification tasks, with up to a 6.97% performance improvement. Moreover, TraGe exhibits superior robustness under parameter fluctuations and variations in sampling configurations.

Index Terms—traffic classification, pre-training technology, generic packet representation, header-payload differences.

I. INTRODUCTION

Traffic classification aims to organize network traffic into different categories (e.g., application and service), which is a fundamental and vital technology in network management and Quality of Service (QoS) [1]–[6]. For instance, in an enterprise network, traffic may include email, voice calls, and file transfers. By effectively classifying these traffic categories, network administrators can implement QoS policies to ensure critical traffic like voice calls receive sufficient bandwidth, low latency, and prioritized network resources [7], [8].

Traditional statistical feature-based methods [9]–[12], such as FlowPrint [9] and AppScanner [10], rely on manual traffic feature extraction, where the extracted features are then fed into machine learning models. In contrast, deep learning-based

methods [13]–[15] automate the learning of traffic features through deep learning models, reducing the need for manual feature extraction. However, deep learning models depend heavily on large-scale labeled data. To address this, some recent pre-trained models [5], [6], [16], [17] leverage large-scale unlabeled raw byte sequences from packets, rather than extracted statistical features, to learn generic representations, requiring only small-scale labeled data for fine-tuning and thereby reducing the reliance on both feature extraction and large-scale labels.

However, existing pre-trained models mainly apply pre-training tasks originating from other domains, such as computer vision (CV) and natural language processing (NLP). These pre-trained tasks are tailored to domain-specific data formats—image data in CV and text data in NLP—which differ significantly from the traffic data in the network domain, making them less effective in learning a generic representation for network traffic. A network packet comprises two fundamentally components: the header and the payload. In contrast to image or text data, which are typically modeled as uniform data, the header and payload exhibit significant structural differences. Specifically, unlike the payload, the header contains structured protocol fields with continuous byte sequences that are essential for recognizing traffic patterns. For instance, the TCP sequence number can reveal program behavior and support anomaly detection [18]. Yet, existing pre-trained models overlook the header byte continuity by either classifying traffic based solely on the payload (e.g., PERT [16], ET-BERT [5]) or by treating the header and payload uniformly, thus disrupting this continuity (e.g., NetGPT [17], YaTC [6]).

To address the above limitations, in this paper, we propose TraGe, a novel pre-trained model for traffic classification with a generic packet representation. This representation is derived by fully considering the header-payload differences of network packets. Specifically, TraGe performs differentiated pre-training based on the byte sequence variations (continuous vs. discontinuous) between the header and payload, with a dynamic masking strategy applied to avoid overfitting to fixed byte positions. Together, these components enable TraGe to obtain a generic packet representation, which can then be fine-tuned for various traffic classification tasks with limited labeled data. Extensive experiments conducted on publicly

^{||} Both authors contributed equally to this research.

^{*} Corresponding author.

available datasets demonstrate the effectiveness and the robustness of TraGe. The comparison results show that TraGe outperforms state-of-the-art pre-trained models, achieving up to a 6.97% improvement in classification performance. In addition, analyses of parameter sensitivity and sampling variability demonstrate TraGe’s robustness to both parameter fluctuations and variations in sampling configurations.

The main contributions of this paper are as follows:

- We propose TraGe, a novel pre-trained model with a generic packet representation for traffic classification based on header-payload differences.
- We design two pre-training tasks (i.e., MLM-FM and MLM-RM) for the header and payload of network packets to obtain generic packet representations.
- We conduct extensive experiments on two traffic classification tasks to evaluate the effectiveness and the robustness of TraGe.

II. RELATED WORK

According to the utilized features, model structure, and pre-training technology, related works can be divided into methods based on statistical features, deep learning, and pre-training technology.

A. Methods based on Statistics Features

Statistical feature-based methods rely on manually extracting traffic features for model training. For example, FlowPrint [19] uses packet inter-arrival times and applies clustering with cross-correlation for classification. AppScanner [10] trains a random forest classifier using packet size statistics, while DTree [11] incorporates size and timing features in a decision tree model. Although these methods demonstrate reasonable effectiveness, their reliance on handcrafted features restricts the model’s ability to learn comprehensive network patterns.

B. Methods based on Deep Learning

Deep learning-based methods automate feature extraction by directly processing raw traffic data through neural networks, eliminating the need for handcrafted features. For example, FS-Net [13] uses recurrent neural networks (RNNs) to learn representations from raw packet length sequences. EBSNN [14] also leverages RNNs but instead operates on raw byte sequences, with two variants: EBSNN-L (based on LSTM) and EBSNN-G (based on GRU). TFE-GNN [15] further explores this direction by modeling byte sequences as graphs and applying graph neural networks (GNNs) for classification. Despite their effectiveness, these methods typically demand large amounts of labeled data, which is often impractical in real-world deployment scenarios [5], [16], [20].

C. Methods based on Pre-training Technology

Pre-trained models leverage self-supervised learning on large-scale unlabeled traffic to learn generalizable representations, which are then fine-tuned using a smaller set of task-specific labeled data. This strategy reduces reliance on extensive annotations by enabling effective classification with

limited labeled samples. For example, ET-BERT [5] treats payload byte sequences as word-like inputs and applies two NLP-inspired pre-training tasks: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). Likewise, YaTC [6] models packet byte sequences—including headers and payloads—as images and uses Masked Image Modeling (MIM), a task from the computer vision domain, to capture inter-traffic relationships. However, current pre-trained models predominantly adopt pre-training tasks from CV and NLP, which are designed for structured image or text data. These data types differ significantly from network traffic, limiting the effectiveness of such tasks in capturing traffic-specific representations.

III. METHODOLOGY

We propose TraGe to fully exploit the traffic characteristics for learning general packet representations, thereby improving model performance in different network traffic classification tasks. Specifically, TraGe leverages the explicit byte distribution differences between the header and the payload to perform header-payload differentiated pre-training (Section III-A). This approach enables the model to obtain a generic packet representation, which facilitates effective model fine-tuning (Section III-B) across diverse traffic classification tasks. The overall framework of TraGe is shown in Fig. 1.

A. Header-Payload Differentiated Pre-training

The header and payload of a packet both consist of byte sequences, but the same byte sequence can represent different meanings. For instance, as shown in Fig. 1, the byte sequence “b11eac20”¹ denotes a sequence number in the header, indicating the position of the packet within a TCP flow. In contrast, in the payload, “b11eac20” could represent a specific data value or instruction, depending on the context of the application and protocol. If identical byte sequences in the header and payload are processed uniformly without considering their distinct meanings, the model may struggle to correctly extract relevant information, which can hinder its learning performance.

Given the differences between the header and payload, we propose header-payload differentiated pre-training based on the byte distribution (continuous vs. discontinuous), as shown in Fig. 1. This approach introduces two pre-training tasks—MLM-FM for the header and MLM-RM for the payload—which together form the foundation of the final generic packet representation. To further enhance pre-training effectiveness, we incorporate a Dynamic Masking (DM) strategy to mitigate model overfitting.

a) Pre-training Task for Header: The network packet header comprises protocol header fields, with each field consisting of *continuous* byte sequences. Existing pre-training models primarily employ Masked Language Modeling (MLM) pre-training task [5], [16], where random byte sequences are masked during model pre-training. The model then learns to predict these masked bytes, thereby forming a generic

¹We represent byte sequences in lowercase hexadecimal format.

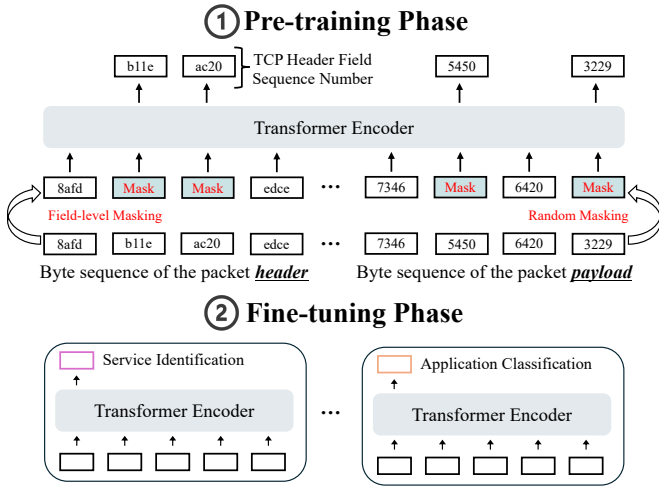


Fig. 1. The framework of TraGe.

packet representation. However, the random masking approach disrupts the continuity of the header fields. For example, a byte sequence like “b11eac20” may represent specific network information, such as a sequence number. Random masking could mask only part of the sequence (e.g., just the byte “b1”), preventing the model from fully capturing the intended network context. As a result, the model’s ability to learn a generic packet representation is compromised.

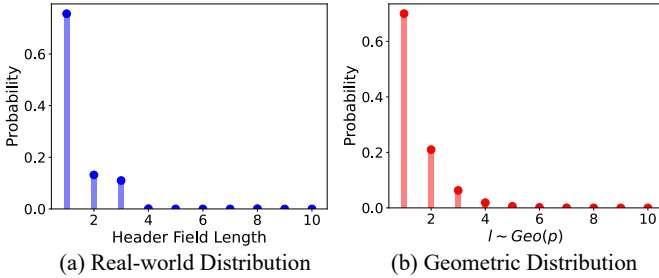


Fig. 2. A case study on header field length distribution on the ISCX-VPN dataset and geometric distribution.

Therefore, we introduce field-level masking to continuously mask byte sequences, thereby improving the model’s ability to learn generic packet representations. A feasible approach to implementing field-level masking is to parse each packet’s header in advance to identify the location of each protocol header field, followed by masking the relevant fields. However, parsing packets can be computationally expensive, and not all protocol headers are easily parsed. To address this, we propose an alternative approach using a geometric distribution. Specifically, we sample the length of the masked byte sequence based on a geometric distribution, drawing inspiration from the observed similarity between the length distribution of header fields and a geometric distribution. As shown in Fig. 2, the distribution of header protocol field lengths in the ISCX-VPN [21] dataset closely aligns with the geometric distribution $\ell \sim \text{Geo}(p)$. This similarity supports the feasibility of using a

geometric distribution for field-level masking.²

During the pre-training of the transformer encoder for the packet header, we first sample a length ℓ using a geometric distribution $\ell \sim \text{Geo}(p)$. Next, we continuously select ℓ tokens from the input sequence, with each token consisting of two consecutive bytes, following the approach in ET-BERT [5]. As shown in Fig. 1, for packet # k , these tokens are masked with the special token [MASK], and the transformer encoder is trained by predicting the masked tokens based on the context. The loss function is based on negative log-likelihood, formally defined as:

$$L_{MLM-FM} = - \sum_{i=1}^{\ell} \log(P(\text{MASK}_i = \text{token}_i | \theta_H)) \quad (1)$$

where θ_H represents the model parameters of the encoder. We pre-train the transformer encoder through Masked Language Modeling (MLM) based on Field-level Masking (FM).

b) Pre-training Task for Payload: Unlike the header, the payload lacks explicit continuity between its bytes. This is primarily due to the nature of the payload, which often carries application-layer data or encrypted content. During transmission, this data is typically fragmented, rearranged, or encrypted, causing the byte order to lose its direct connection to the original data structure. Furthermore, encryption disrupts the continuity between bytes, increasing processing complexity. Therefore, we directly use Masked Language Modeling based on Random Masking (MLM-RM). Compared with field-level masking, random masking randomly selects r tokens in the input token sequence. The loss function is defined as:

$$L_{MLM-RM} = - \sum_{i=1}^r \log(P(\text{MASK}_i = \text{token}_i | \theta_P)) \quad (2)$$

where θ_P represents the set of parameters of the transformer encoder. RM means Random Masking.

c) Dynamic Masking for Model Pretraining: Traditional pre-training models typically adopt static masking, where a fixed subset of tokens is selected for prediction during the data preprocessing stage [5], [16]. These masked positions remain unchanged throughout model pre-training, leading the model to repeatedly encounter the same masking pattern. This approach restricts the model’s exposure to diverse traffic information and encourages memorization of specific positions rather than learning generic traffic representations, ultimately limiting generalization. To address this limitation, we adopt a dynamic masking strategy that generates masked positions dynamically during model pre-training. This approach allows the model to observe varied byte combinations across training iterations, thereby improving the generalization and robustness of the learned traffic representations.

B. Model Fine-tuning

The objective of model fine-tuning is to classify a given network flow into a specific traffic category. Based on the

²We evaluate the impact of the geometric distribution parameters on the field-level masking in Section IV-D. The experimental results show insignificant performance changes across these parameters.

obtained generic packet representation, we can perform different traffic classification tasks. We utilize multiple linear layers and the softmax function to predict the probabilities of the network flow belonging to different traffic categories. Note that there is no need to design separate models for each traffic classification task. This is because traffic characteristics can be fully exploited through header-payload differentiated pre-training to generate a generic packet representation, which then supports various traffic classification tasks.

IV. EVALUATION

In this section, we first introduce the selected datasets, baselines, and the detailed implementation of TraGe. Subsequently, we evaluate TraGe on two traffic classification tasks. Our evaluation aims to answer four research questions:

RQ1. Can TraGe outperform state-of-the-art (SoTA) traffic classification models?

RQ2. What contribution does each TraGe component make?

RQ3. How do parameter changes affect TraGe's classification performance?

RQ4. How does data sampling variability affect the performance stability of TraGe?

A. Experiment Setup

a) *Datasets:* We select three datasets from diverse sources for pre-training and fine-tuning, including ISCX-VPN [21], USTC-TFC [22], and CIC-IoT [23]. The ISCX-VPN dataset captures traffic from various behaviors across multiple applications and services within VPN environments, such as browsing, email, audio, video, and file transfer. The USTC-TFC dataset contains 10 categories of benign traffic and 10 categories of malicious traffic, primarily from smart grid infrastructures. The CIC-IoT dataset is an IoT traffic dataset designed for behavioral analysis and intrusion detection.

We use a portion of these datasets for model pre-training and the remaining data for model fine-tuning. Two traffic classification tasks on the ISCX-VPN dataset are performed, i.e., application classification and service identification. The application classification and service identification tasks focus on encrypted traffic in VPN environments. To evaluate the model's ability to classify applications and services in detail, we further categorize the ISCX-VPN dataset according to applications and services, i.e., ISCX-VPN (App) and ISCX-VPN (Service), ultimately forming a 17-class application classification task and a 12-class service identification task. Table I presents the specific traffic category distribution for the two traffic classification tasks.

b) *Implementation Details and Baselines:* We implement TraGe and all baselines by using Python 3.10.14 with libraries including NumPy 1.26.14, PyTorch 2.3.0, and CUDA 12.4. For model pre-training, we use BERT [24] as the transformer encoder. We perform pre-training for 100,000 steps, with a learning rate of $1e-3$. During field-level masking, we apply a geometric distribution with parameters $p = 0.7$. Here, p controls the shape of the distribution. During model fine-tuning, we set the number of epochs to 10, with a learning

TABLE I
THE CATEGORY INFORMATION OF THE FINE-TUNING TASKS.

Task	Traffic Category	#label
Application Classification	ftp, spotify, vimeo, hangout, ICQ, netflix, gmail, AIM, YouTube, voipbuster, sftp	17
	email-client, scp, skype, Torrent, Facebook, Tor	
Service Identification	Chat, VPN-Chat, VPN-P2P, P2P, Voip, VPN-Voip, FT, VPN-FT, Email, VPN-Email, Streaming, VPN-Streaming	12

rate of $2e-5$. In the stage of model fine-tuning, we select a maximum of 5 packets per flow for traffic classification. We randomly select at most 5000 flows from each traffic category for all tasks. Each sampled dataset is then divided into the training set, the validation set, and the testing set according to the ratio of 8 : 1 : 1.

To ensure a fair comparison, we use three typical metrics, i.e., Precision, Recall, and F1-Score. We select twelve baselines, including (1) four statistics features based models, i.e., FlowPrint [9], AppScanner [10], XGBoost [12], and DTree [11]; (2) four deep learning based models, i.e., FS-Net [13], EBSNN-L [14], EBSNN-G [14], TFE-GNN [15]; and (3) four pre-trained models, i.e., PERT [16], ET-BERT [5], NetGPT [17], and YaTC [6].

B. Overall Classification Performance (RQ1)

We show the classification performance of TraGe and all baselines on two traffic classification tasks in TABLE II and TABLE III. Bold text indicates the best results, and underlined text indicates the second-best results among all models. It can be seen that TraGe outperforms all baselines with significant margins across both tasks. As shown in Table II, it obtains a Precision of 0.7517, a Recall of 0.7541, and an F1-Score of 0.7484 on the application classification task. Similarly, Table III shows that TraGe achieves a Precision of 0.9333, a Recall of 0.9335, and an F1-Score of 0.9331 on the service identification task. These results surpass the second-best results (0.7421, 0.7187, 0.7182, 0.9250, 0.9192, and 0.9207) by 1.29%, 4.93%, 4.20%, 0.90%, 1.56%, and 1.35%, respectively. Besides, we observe that the second-best results are all achieved by pre-trained models rather than statistics features-based and deep learning-based models. Further, the results show that TraGe outperforms these four pre-trained models, with average performance improvements of 4.21%, 4.08%, 2.57%, and 6.97% on two tasks. This is because TraGe fully exploits the characteristics of network traffic data to learn a generic packet representation, thereby supporting the best classification results on different traffic classification tasks. In contrast, the baselines fail to fully account for these traffic characteristics, resulting in less generic representations that limit their classification performance.

TABLE II
CLASSIFICATION PERFORMANCE COMPARISON OF DIFFERENT METHODS
ON APPLICATION CLASSIFICATION TASK.

Method	Precision	Recall	F1-Score
FlowPrint [25]	0.5904	0.4304	0.4494
AppScanner [10]	0.7289	0.5361	0.5803
XGBoost [12]	0.5177	0.4151	0.4304
DTree [11]	0.4844	0.4483	0.4462
FS-Net [13]	0.4990	0.3996	0.4060
EBSNN-L [14]	0.7176	0.6691	0.6781
EBSNN-G [14]	0.7285	0.6780	0.6882
TFE-GNN [15]	0.6720	0.6060	0.6180
PERT [16]	0.7218	0.6971	0.6955
ET-BERT [5]	0.7118	0.7027	0.6990
NetGPT [17]	0.7336	0.7187	0.7182
YaTC [6]	0.7421	0.6746	0.6870
TraGe	0.7517	0.7541	0.7484

TABLE III
CLASSIFICATION PERFORMANCE COMPARISON OF DIFFERENT METHODS
ON SERVICE IDENTIFICATION TASK.

Method	Precision	Recall	F1-Score
FlowPrint [25]	0.7021	0.6662	0.6451
AppScanner [10]	0.8599	0.7567	0.7913
XGBoost [12]	0.7094	0.7160	0.7032
DTree [11]	0.7026	0.7090	0.6997
FS-Net [13]	0.7161	0.6363	0.6418
EBSNN-L [14]	0.9000	0.8672	0.8771
EBSNN-G [14]	0.9013	0.8784	0.8845
TFE-GNN [15]	0.8597	0.8095	0.8214
PERT [16]	0.9213	0.9148	0.9149
ET-BERT [5]	0.9214	0.9182	0.9191
NetGPT [17]	0.9250	0.9192	0.9207
YaTC [6]	0.9053	0.8565	0.8663
TraGe	0.9333	0.9335	0.9331

C. Ablation Study (RQ2)

In this subsection, we present the results of an ablation study of TraGe on the application classification and service identification tasks, as shown in TABLE IV. To streamline the presentation, we use the following abbreviations: ‘FM’ for field-level masking, ‘DM’ for dynamic masking. When FM is not applied, random masking is used by default. Similarly, in the absence of DM, static masking is employed.

As shown in TABLE IV, the results for both the application classification and the service identification tasks show that removing either field-level masking or dynamic masking significantly degrades model performance. Specifically, eliminating field-level masking leads to a 5.88% decrease in F1-score for the application classification task, while removing dynamic masking causes a 1.62% reduction. Similar trends are observed in both Precision and Recall. Overall, the results demonstrate that TraGe’s header-payload differentiated pre-training enables the formation of a generic packet representation, which in turn improves model classification performance across different traffic classification tasks.

TABLE IV
ABLATION STUDY OF KEY COMPONENTS IN TRA-GE ON THE APPLICATION
CLASSIFICATION AND SERVICE IDENTIFICATION TASKS.

Method	Application Classification		
	Precision	Recall	F1-Score
Ours (TraGe)	0.7517	0.7541	0.7484
Ours w/o FM	0.7318	0.7044	0.7044
Ours w/o DM	0.7489	0.7349	0.7363
Ours w/o FM & DM	0.7234	0.7106	0.7086

Method	Service Identification		
	Precision	Recall	F1-Score
Ours (TraGe)	0.9333	0.9335	0.9331
Ours w/o FM	0.9275	0.9226	0.9242
Ours w/o DM	0.9316	0.9302	0.9305
Ours w/o FM & DM	0.9241	0.9240	0.9236

D. Parameter Sensitivity Analysis (RQ3)

In this subsection, we evaluate the impact of various parameter settings in the header-payload differentiated pre-training on the effectiveness of TraGe. In the pre-training of the transformer encoder, we utilize field-level masking to exploit the continuity of header fields. Specifically, the length of the masked sequence is sampled from a geometric distribution $\ell \sim \text{Geo}(p)$, where the parameter p controls the masking length probability. By adjusting the value of p , we aim to assess how sensitive the model is to the parameter of field-level masking. As shown in Fig. 3, TraGe exhibits stable classification performance on both application classification and service identification tasks across various values of p . For instance, on the service identification task, the F1-Score fluctuates only slightly within the range of 0.9275 to 0.9323, yielding a maximum difference of just 0.0048. These results demonstrate that TraGe is robust to changes in this masking parameter and that its effectiveness does not rely heavily on the configuration of the field-level masking.

E. Sampling Variability Analysis (RQ4)

In this subsection, we evaluate the robustness of TraGe to variations in dataset sampling by evaluating its performance across multiple sampled datasets. To simulate different training conditions, we randomly select up to 5,000 samples from each traffic category using a range of random seeds. Specifically, we vary the random seed from 1 to 20, generating 20 distinct training and test set configurations. Fig. 4 presents the classification performance of TraGe under these different sampling datasets. The results show that TraGe maintains stable performance across both the application classification and service identification tasks, regardless of the specific sampling instance. For example, on the service identification task, the F1-Score varies only slightly between 0.9198 and 0.9414, with a maximum difference of just 0.0216. This consistent performance across diverse sampled datasets demonstrates the robustness of TraGe to changes in sampling settings.

V. CONCLUSION

In this paper, we propose TraGe, a novel traffic classification model based on pre-training technology. Targeting

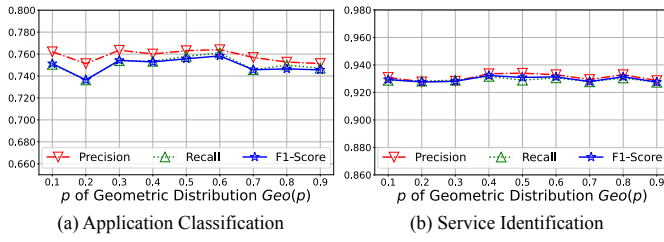


Fig. 3. Parameter analysis on field-level masking.

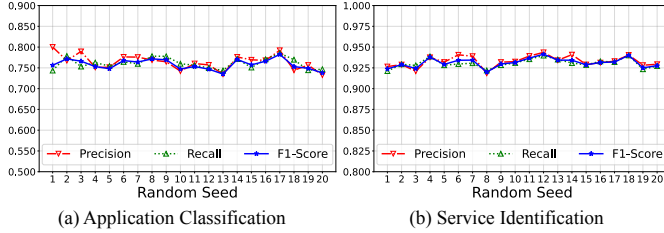


Fig. 4. Classification Performance under different sampling datasets.

the header-payload differences, TraGe effectively forms a generic packet representation adaptable to various traffic classification tasks. An overall comparison, along with detailed evaluations—encompassing ablation study, parameter sensitivity analysis, and sampling variability analysis—is provided, demonstrating the effectiveness of TraGe. The experimental results show that TraGe outperforms state-of-the-art methods on two traffic classification tasks. Moreover, TraGe demonstrates strong robustness in both parameter sensitivity and sampling variability analyses.

ACKNOWLEDGMENT

This work was supported in whole or in part, by National Natural Science Foundation of China (62372429, U24B6012 and U2333201), the Innovation Funding of ICT, CAS under Grant No. E461040, Pilot for Major Scientific Research Facility of Jiangsu Province of China (No.BM2021800).

REFERENCES

- [1] M. Shen, Z. Gao, L. Zhu, and K. Xu, "Efficient fine-grained website fingerprinting via encrypted traffic analysis with deep learning," in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. IEEE, 2021, pp. 1–10.
- [2] L. Yang, Y. Wang, L. Liu, J. Huang, and S. Fu, "Expmd: an explainable framework for traffic identification based on multi-domain features," in *2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQOS)*. IEEE, 2024, pp. 1–10.
- [3] K. Mao, X. Xiao, G. Hu, X. Luo, B. Zhang, and S. Xia, "Byte-label joint attention learning for packet-grained network traffic classification," in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*. IEEE, 2021, pp. 1–10.
- [4] W. Wang, S. Zhu, Z. Wu, L. Lu, Z. Li, H. Yang, and Y. Zhang, "Radd: A real-time and accurate method for ddos detection based on in-network computing," in *ICC 2024-IEEE International Conference on Communications*. IEEE, 2024, pp. 3316–3321.
- [5] X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu, "Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification," in *Proceedings of the ACM Web Conference 2022*, 2022, pp. 633–642.
- [6] R. Zhao, M. Zhan, X. Deng, F. Li, Y. Wang, Y. Wang, G. Gui, and Z. Xue, "A novel self-supervised framework based on masked autoencoder for traffic classification," *IEEE/ACM Transactions on Networking*, 2024.

- [7] M. Karakus and A. Durrezi, "Quality of service (qos) in software defined networking (sdn): A survey," *Journal of Network and Computer Applications*, vol. 80, pp. 200–218, 2017.
- [8] L. Wang, M. Wang, C. Lin, and Y. Zhang, "Accelerating traffic engineering optimization for segment routing: A recommendation perspective," *Computer Networks*, vol. 264, p. 111224, 2025.
- [9] T. Van Ede, R. Bortolameotti, A. Continella, J. Ren, D. J. Dubois, M. Lindorfer, D. Hoffnes, M. van Steen, and A. Peter, "Flowprint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Network and distributed system security symposium (NDSS)*, vol. 27, 2020.
- [10] V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic, "Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic," in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, pp. 439–454.
- [11] J. Dai, Y. Chen, Y. Chen, and A. Meng, "An analysis of network traffic identification based on decision tree," in *2021 International Conference on Artificial Intelligence and Electromechanical Automation (AIEA)*. IEEE, 2021, pp. 308–311.
- [12] I. L. Cherif and A. Kortebe, "On using extreme gradient boosting (xgboost) machine learning algorithm for home network traffic classification," in *2019 Wireless Days (WD)*. IEEE, 2019, pp. 1–6.
- [13] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "Fs-net: A flow sequence network for encrypted traffic classification," in *IEEE INFOCOM 2019-IEEE Conference On Computer Communications*. IEEE, 2019, pp. 1171–1179.
- [14] X. Xiao, W. Xiao, R. Li, X. Luo, H. Zheng, and S. Xia, "Ebsnn: Extended byte segment neural network for network traffic classification," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3521–3538, 2021.
- [15] H. Zhang, L. Yu, X. Xiao, Q. Li, F. Mercaldo, X. Luo, and Q. Liu, "Tfe-gnn: A temporal fusion encoder using graph neural networks for fine-grained encrypted traffic classification," in *Proceedings of the ACM Web Conference 2023*, 2023, pp. 2066–2075.
- [16] H. Y. He, Z. G. Yang, and X. N. Chen, "Pert: Payload encoding representation from transformer for encrypted traffic classification," in *2020 ITU Kaleidoscope: Industry-Driven Digital Transformation (ITU K)*. IEEE, 2020, pp. 1–8.
- [17] X. Meng, C. Lin, Y. Wang, and Y. Zhang, "Netgpt: Generative pretrained transformer for network traffic," *arXiv preprint arXiv:2304.09513*, 2023.
- [18] S. Zhang, M. Gao, L. Wang, S. Xu, W. Shao, and R. Kuang, "A malware-detection method using deep learning to fully extract api sequence features," *Electronics*, vol. 14, no. 1, p. 167, 2025.
- [19] E. Horowicz, T. Shapira, and Y. Shavitt, "A few shots traffic classification with mini-flowpic augmentations," in *Proceedings of the 22nd ACM Internet Measurement Conference*, 2022, pp. 647–654.
- [20] A. Bahramali, A. Bozorgi, and A. Houmansadr, "Realistic website fingerprinting by augmenting network traces," in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 1035–1049.
- [21] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and vpn traffic using time-related," in *Proceedings of the 2nd international conference on information systems security and privacy (ICISSP)*, 2016, pp. 407–414.
- [22] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *2017 International conference on information networking (ICOIN)*. IEEE, 2017, pp. 712–717.
- [23] S. Dadkhah, H. Mahdikhani, P. K. Danso, A. Zohourian, K. A. Truong, and A. A. Ghorbani, "Towards the development of a realistic multi-dimensional iot profiling dataset," in *2022 19th Annual International Conference on Privacy, Security & Trust (PST)*. IEEE, 2022, pp. 1–11.
- [24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [25] X. Han, G. Xu, M. Zhang, Z. Yang, Z. Yu, W. Huang, and C. Meng, "Degnn: Dual embedding with graph neural network for fine-grained encrypted traffic classification," *Computer Networks*, vol. 245, p. 110372, 2024.